

Computer Organization, Evolution & Hardware Systems

1. COMPUTER ARCHITECTURE VS. ORGANIZATION

A clear boundary separates these two core concepts in computer systems engineering:

- **Computer Architecture:** Refers to those operational parameters of a computer system that are directly visible to an assembly programmer.

Examples: The Instruction Set Architecture (ISA), CPU register configurations, memory addressing schemes, and data types.

- **Computer Organization:** Refers to the underlying physical hardware operational units and their interconnections that realize the architectural specifications.

Examples: Control signals, memory bus technology, interface logic, and whether SRAM or DRAM is chosen for primary storage.

2. EVOLUTION OF DIGITAL COMPUTERS

Computer evolution is categorized into generations based on the fundamental electronic switches used:

Gen (Years)	Core Electronics	Famous Examples
1st (1945–1955)	Vacuum Tubes	ENIAC, EDVAC
2nd (1955–1965)	Discrete Transistors	IBM 7094, CDC 1604
3rd (1965–1980)	Integrated Circuits	IBM System/360
4th (1980–Present)	VLSI Microprocessors	Intel x86, ARM
5th (Future)	ULSI / Parallel / AI	Quantum / Neural Chips

The Size Reduction Breakthrough

First-generation machines like the **ENIAC** used over 18,000 physical **Vacuum Tubes**. They were massive (occupying whole rooms), generated extreme heat, and failed constantly because tubes burned out.

The invention of the solid-state **Transistor** and integrated micro-**Capacitors** in the 2nd and 3rd generations changed everything.

- Transistors acted as binary switches (0 and 1) but were thousands of times smaller, consumed minimal power, and did not burn out.
- Capacitors allowed tiny electrical charges to be stored reliably to represent data.

This transition allowed billions of switches to be packed onto a single silicon chip, causing computer size to shrink from room-sized to hand-held.

3. MOORE'S LAW

Coined by Intel co-founder Gordon Moore, **Moore's Law** is an empirical observation stating that:

"The number of transistors packed onto a microchip doubles roughly every two years, while the cost of computers is halved."

This law drove the exponential growth of computing power, smaller device sizes, and lower consumer costs for decades.

4. MEMORY HIERARCHY

CPUs process data at extreme speeds. To keep the CPU fed with data efficiently without spending too much money on expensive components, computer designers utilize a **Memory Hierarchy**:

```
▲ [Fastest, Most Expensive, Lowest Capacity]
/ \   Level 0: CPU Registers (Inside CPU)
/ \   Level 1: L1 / L2 / L3 Cache (SRAM)
/ \   Level 2: Primary Memory (DRAM RAM)
/=====\ Level 3: Secondary Storage (SSD, HDD)
/-----\ [Slowest, Cheapest, Highest Capacity]
```

5. RAM TECHNOLOGIES: SRAM VS. DRAM

Primary storage is divided into two distinct solid-state semiconductor technologies:

Feature	SRAM (Static RAM)	DRAM (Dynamic RAM)
Core Unit	Uses Flip-Flops (6 transistors per bit).	Uses 1 transistor and 1 capacitor per bit.
Refresh	Does not need refreshing.	Needs constant refreshing (charge leaks).
Speed	Extremely Fast (1-2 ns).	Slower (50-100 ns).
Cost/Density	Highly expensive, low density.	Cheap, very high density.
Usage	CPU Cache (L1, L2, L3).	Main System RAM.

6. CENTRAL PROCESSING UNIT (CPU) CORE ELEMENTS

The CPU contains two fundamental logic controllers to execute commands:

A. Arithmetic Logic Unit (ALU)

The calculator of the processor. It takes numerical inputs from registers and performs:

- **Arithmetic operations:** Addition, subtraction, multiplication.
- **Logical operations:** Bitwise AND, OR, NOT, XOR, and comparisons.

B. Control Unit (CU)

The traffic cop of the CPU. It directs the flow of data. It fetches instructions, decodes them, and coordinates other parts of the system by sending out synchronized control signals (such as read/write and ALU operational signals).

7. STORAGE MEDIA TECHNOLOGIES

- **Magnetic Storage:** Uses magnetic fields to align domains on a physical disk or tape (e.g., HDDs, Magnetic Tapes). Non-volatile, cheap, but slower due to physical spinning parts.
- **ROM (Read-Only Memory):** Non-volatile memory that cannot be easily written to. Used to hold boot-up firmware (BIOS/UEFI).
- **Optical Storage (DVD, CD, Blu-ray):** Uses laser beams to read and write data written as tiny physical microscopic "pits" (dark spots) and "lands" (reflective flat surfaces) on a plastic disc.
- **SSD (Solid State Drives):** High-speed secondary storage using non-volatile flash memory chips. Has no moving parts, making them incredibly fast.

8. SAMPLE EXAM QUESTIONS

Q1. Explain why DRAM requires a periodic refresh cycle while SRAM does not.

Answer: DRAM stores bits as electrical charges inside tiny capacitors. Capacitors naturally leak charge over time. To prevent data from being lost, DRAM must be read and rewritten (refreshed) hundreds of times per second. SRAM uses transistor flip-flops, which lock their state continuously as long as power is applied.

Q2. How did the transition from Vacuum Tubes to Transistors influence computer design?

Answer: Vacuum tubes were bulky, fragile, consumed high power, and produced extreme heat. Transistors replaced them as solid-state binary switches that were much smaller, highly reliable, and energy-efficient. This allowed engineers to place millions of logic gates in small spaces, reducing computer sizes dramatically.

The Multilevel Machine & Translation Methods

1. CONCEPT OF VIRTUAL MACHINES & INTERPRETATION

A physical digital computer understands only native binary machine language (0s and 1s), which is extremely complex for humans to read or write.

To resolve this, computer systems are structured as a series of **virtual machines**, built hierarchically. Each level uses a language more convenient than the one below it.

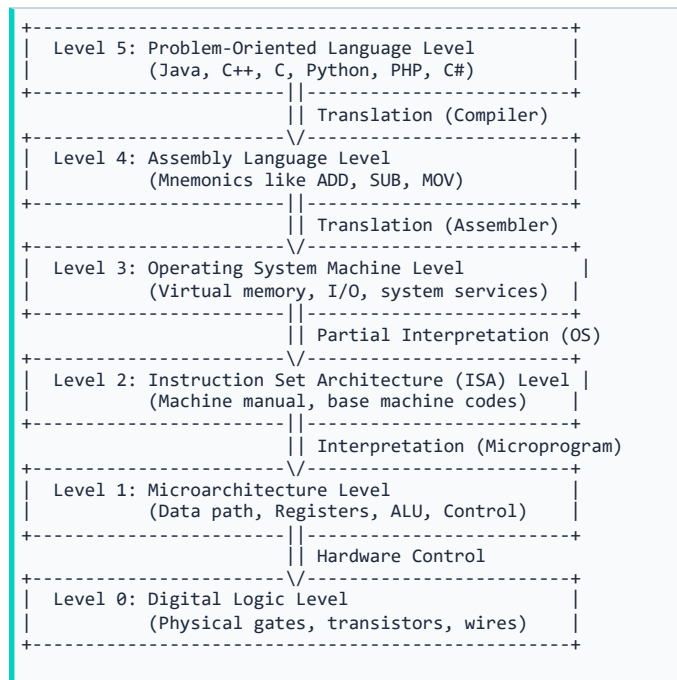
Translation vs. Interpretation

To execute a program written in a high-level language (\$L_r\$) on a machine that only runs a low-level language (\$L_0\$), we use two primary methods:

- **Translation (Compilation):** The entire program in \$L_r\$ is translated *before* execution into an equivalent program written in \$L_0\$. The original program is discarded, and the new \$L_0\$ program runs directly on the hardware.
- **Interpretation:** No pre-translation occurs. Instead, a special program called an **interpreter** reads each instruction of \$L_r\$ one-by-one, translates it into \$L_0\$ instructions on-the-fly, and executes it immediately.

2. THE SIX-LEVEL COMPUTER HIERARCHY

Modern computer architectures are organized into six distinct layers of abstraction:



Deep-Dive of Each Level

- **Level 0: Digital Logic Level:** The actual physical hardware. Composed of silicon transistors, physical wires, logic gates (AND, OR, NOT), and bistable memory flip-flops.
- **Level 1: Microarchitecture Level:** Contains the **Data Path** (the ALU and internal CPU registers R0-R15) and a Control Unit. In some CPUs, a local microprogram interprets Level 2 instructions by controlling the data path.
- **Level 2: Instruction Set Architecture (ISA):** The base machine language level. This is the official interface/instruction manual of the CPU (e.g., x86 or ARM instruction manuals).
- **Level 3: Operating System Level:** Built directly on the ISA. It handles file systems, memory protection, virtual memory, and process control.
- **Level 4: Assembly Language:** Human-readable version of machine language. Uses text mnemonics (e.g., add r1, r2) instead of binary bit patterns.
- **Level 5: Problem-Oriented Language:** Standard high-level languages used by application programmers to solve problems easily.

3. COMPILER VS. INTERPRETER

These two translation systems have contrasting operational mechanics and trade-offs:

Feature	Compiler (Translation)	Interpreter (Direct)
Mechanism	Translates the entire source code into binary at once.	Translates and runs source code line-by-line.
Speed	Fast execution (already fully compiled).	Slower execution (translates on-the-fly).
Memory	Generates temporary object files (.obj, .exe).	No intermediate files created. Less storage.
Debugging	Harder; reports all syntax errors at the end.	Easier; stops immediately on the error line.

Compiler Examples & Why They are Used:

- **C & C++:** Highly compiled languages. They convert source code directly into native machine code of the target operating system and CPU (e.g., .exe in Windows).
Why: Used for Operating Systems (Windows, Linux), game engines, and performance-critical systems because there is no runtime translation overhead, ensuring maximum execution speed.

Interpreter Examples & Why They are Used:

- **Python, PHP, and JavaScript:** Interpret code directly.
Why: Used for rapid web development and scripting because programmers can write code and test it instantly without waiting for a lengthy compilation step. It is highly cross-platform, as long as the target machine has the interpreter installed.

The Hybrid Approach (e.g., Java):

Java combines both methods. The Java Compiler translates source code (.java) into an intermediate, platform-neutral format called **Bytecode** (.class).

During execution, the **Java Virtual Machine (JVM)** interprets this Bytecode (or uses a Just-In-Time Compiler) to convert it into target-machine binary code. This delivers the famous feature: "Write Once, Run Anywhere."

4. SELF-ASSESSMENT & SCENARIOS

Q1. Write down the sequence of translation steps that occur when a programmer executes a high-level source file down to physical hardware.

Answer:

1. **High-Level Code (L5):** Written in C.
2. **Compiler:** Translates C code into Assembly (L4).
3. **Assembler:** Translates Assembly into ISA Machine Binary (L2).
4. **OS (L3):** Loads the binary file into memory, resolves system calls, and requests execution.
5. **Microarchitecture (L1):** The CPU's microprogrammed control unit translates machine instructions into control paths, switching physical transistors and gates (L0) on and off.

Q2. Why is Level 3 (OS Level) considered a "Hybrid" level?

Answer: It is considered a hybrid because most OS commands are identical to Level 2 (ISA) instructions and pass directly to lower levels without translation. However, a small subset of instructions (like I/O handling, memory management, and file operations) are intercepted and interpreted by the OS itself.

Q3. A company is building a self-driving car algorithm. Would they write this program using an interpreted language (like Python) or a compiled language (like C++)? Explain your choice.

Answer: They would write it in a compiled language like C++. Self-driving car calculations (image recognition, obstacle avoidance) must happen in real-time within microseconds. An interpreted language like Python has too much runtime translation overhead, which could cause hazardous delays. Compiled C++ code runs directly on the CPU hardware at native speed.

Basic Computer Operations & Bus Architectures

1. PRIMARY MEMORY INTERFACE REGISTERS

The CPU and the Primary Memory (RAM) communicate through a dedicated system bus. To manage this interface, the CPU uses two crucial special-purpose registers:

- **Memory Address Register (MAR):** Holds the exact physical address of the memory location that the CPU wants to read from or write to.
- **Memory Data Register (MDR):** Holds the actual data. During a **Read** operation, it receives data fetched from memory. During a **Write** operation, it holds the data waiting to be written into memory.

The Interfacing Process

Memory is represented as a linear array of locations, each with a unique address.



2. INTERNAL CPU COMPONENTS & REGISTERS

Inside the processor, several core components handle instruction execution:

A. Instruction Register (IR)

Holds the binary code of the instruction currently being executed. Once an instruction is fetched from memory, it is loaded into the IR so that the Control Unit can decode it.

B. Program Counter (PC)

Keeps track of the execution flow by holding the memory address of the next instruction to be fetched and executed.

C. Temporary ALU Registers (Y and Z)

In a single-bus processor design, the ALU cannot fetch two operands from the bus and output the result simultaneously. Thus, temporary registers are needed:

- **Register Y:** Temporarily holds one of the operands for the ALU, keeping it stable while the second operand is loaded from the bus.
- **Register Z:** Temporarily holds the output result generated by the ALU before it is placed back onto the shared bus.

D. Multiplexer (Mux)

A selector circuit that chooses between two inputs to feed into the ALU:

- **Input A:** The value stored in Register Y.
- **Constant 4:** Used to directly increment the Program Counter ($PC \leftarrow PC + 4$) during the fetch phase.

E. Control Unit (CU)

The brain of the CPU. It decodes the instruction in the IR and coordinates all internal components by generating timed **Control Signals** (such as R_{in} , R_{out} , Read, Write).

3. MICRO-OPERATIONS & REGISTER TRANSFERS

Every high-level machine instruction is executed as a sequence of elementary step-by-step actions called **Micro-operations**.

Data moves between registers over the internal CPU bus using gating signals:

- **R_{out} (Output Gate):** Connects a register to the bus, placing its data onto the bus lines.
- **R_{in} (Input Gate):** Connects the bus to a register, loading the current bus data into that register.

How Data Transfers Between IR, MDR, and Memory

Data does not jump directly. It flows systematically across buses:

1. **Memory to MDR (Read):** The CU asserts the Read signal. Memory retrieves data at the address specified by the MAR and places it on the external system data bus, loading it into the MDR.
2. **MDR to IR (Instruction Decode):** Once inside the MDR, the instruction is transferred across the internal processor bus into the IR by asserting MDR_{out} and IR_{in} .

4. INTERNAL PROCESSOR BUS ARCHITECTURES

Processors connect their internal components using different bus layouts, which directly impacts execution speed.

A. Single-Bus Architecture

A single internal shared bus connects all registers, the ALU, and the memory interface.

- **How it works:** Only one data transfer can take place over the bus in any single clock cycle.
- **Why Y and Z matter:** Because there is only one bus, the ALU cannot read two inputs at once. One input must be latched into Register Y. The ALU output must be held in Register Z because the input bus is occupied.
- **Con:** Slow execution due to serialized data transfers.

B. Two-Bus Architecture

Features two separate internal buses (typically named Bus A and Bus B).

- **How it works:** Allows two operands to be transferred from different registers to the ALU inputs simultaneously.
- **Why it matters:** Eliminates the need for Register Y. The ALU can read directly from Bus A and Bus B at the same time.

C. Multi-Bus Architecture

Features three or more independent internal buses (e.g., Bus A, Bus B, and Bus C).

- **Why it matters:** Allows two source operands to be transferred to the ALU, and the ALU output to be transferred to a destination register, all within **one single clock cycle**.
- **Physical Advantage:** Having multiple shorter buses instead of one long bus reduces **parasitic capacitance**, which significantly decreases propagation delay and allows higher clock speeds.

5. STEP-BY-STEP SCENARIO TRACING

Let's look at the exact micro-operations that occur during execution phases.

Scenario A: Fetching an Instruction

The universal sequence to fetch any instruction from memory:

Step	Micro-operations & Control Signals
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in} (Move PC to MAR, initiate memory read, compute $PC+4$, save to Z)
2	Z_{out} , PC_{in} , WMFC (Update PC with $PC+4$, Wait for Memory Function Complete)
3	MDR_{out} , IR_{in} (Transfer fetched instruction from MDR to IR)

Scenario B: Executing ADD R1, R2, R3 ($R1 \leftarrow R2 + R3$)

Assuming the instruction has already been fetched into the IR:

Step	Micro-operations & Control Signals
4	$R2_{out}$, Y_{in} (Move operand 1 from R2 into temporary Register Y)
5	$R3_{out}$, SelectY, Add, Z_{in} (Place R3 on bus, select Y, add them, store result in Z)
6	Z_{out} , $R1_{in}$, End (Write result from Z to destination register R1)

6. SELF-ASSESSMENT PRACTICE QUESTIONS

Q1. Why does a Single-Bus processor require Register Z?

Answer: Since there is only one bus, the output of the ALU cannot be written directly to a destination register in the same cycle that the input operands are occupying the bus. Register Z temporarily buffers the output until the bus is free.

Q2. What is the role of the WMFC control signal?

Answer: WMFC stands for "Wait for Memory Function Complete". Memory units are much slower than CPU registers. This signal forces the processor's control circuitry to pause until the memory unit signals that the read/write operation is finished.

Q3. Explain why parasitic capacitance decreases in Multi-Bus architectures.

Answer: A single bus running across the entire chip is physically long and accumulates high parasitic capacitance, which slows down electrical signals. Multi-bus designs use multiple, shorter localized buses. Shorter physical wires mean less capacitance, causing smaller propagation delays.

1. MEMORY ORGANIZATION OVERVIEW

Memory is a critical sub-system that strongly dictates a computer's overall speed and efficiency.

Conceptual View: Memory can be seen as a large, linear array of storage locations, where each location is associated with a **unique numerical address**.

Specification (\$M \times N\$): A memory system with \$M\$ storage locations and \$N\$ bits per location is referred to as an \$M \times N\$ memory. Both \$M\$ and \$N\$ are typically powers of 2.

Examples:

- 1024×8 (1024 locations, each holding 8 bits)
- 65536×32 (65,536 locations, each holding 32 bits)

Key Data Size Terminologies

- **Bit:** The smallest unit of digital storage (0 or 1).
- **Nibble:** A collection of 4 bits.
- **Byte:** A collection of 8 bits.
- **Word:** The natural unit of data used by a specific CPU architecture. The size varies (typically 16, 32, or 64 bits) and matches the CPU's register size.

2. MEMORY ADDRESSING SCHEMES

Computers read and write data based on addresses. There are two primary addressing designs:

A. Byte-Addressable Memory (Standard)

Every individual **Byte (8 bits)** in the memory has its own unique, distinct address.

If a computer has a 32-bit Word size (which equals 4 Bytes) and is Byte-addressable, then one single Word will span across **4 consecutive memory addresses**.

Example: 32-bit Word starting at Address 0

Address 0: Bytes 0 (bits 0-7)

Address 1: Bytes 1 (bits 8-15)

Address 2: Bytes 2 (bits 16-23)

Address 3: Bytes 3 (bits 24-31)

B. Word-Addressable Memory

Each unique address points to a complete **Word** of data. Individual bytes inside that word do not have their own separate addresses.

If the system is 32-bit Word-addressable:

- Address 0 points to Word 0 (bits 0-31).
- Address 1 points to Word 1 (bits 32-63).

3. BYTE ORDERING (ENDIANNESS)

When storing a multi-byte word (e.g., a 32-bit integer) in a Byte-addressable memory, the bytes must be ordered. There are two main methods:

Let's use the 32-bit Hexadecimal value **0x12345678** (stored starting at memory Address 1000):

- **Most Significant Byte (MSB):** 12
- **Least Significant Byte (LSB):** 78

A. Big Endian Order

The **MSB (most significant byte)** is stored at the lowest (first) memory address.

Address	1000	1001	1002	1003
Data (Hex)	12 (MSB)	34	56	78 (LSB)

Used by: Motorola, SPARC, and internet protocols.

B. Little Endian Order

The **LSB (least significant byte)** is stored at the lowest (first) memory address.

Address	1000	1001	1002	1003
Data (Hex)	78 (LSB)	56	34	12 (MSB)

Used by: Intel x86 architectures.

4. MEMORY ALIGNMENT

In a Byte-addressable system, a data object of size \$s\$ bytes is considered **aligned** if its starting memory address is a multiple of \$s\$.

Alignment rules:

- 2-byte object (Half-word): Must start at an address divisible by 2 (0, 2, 4, 6...).
- 4-byte object (Word): Must start at an address divisible by 4 (0, 4, 8, 12...).
- 8-byte object (Double-word): Must start at an address divisible by 8 (0, 8, 16...).

Performance Implication

Hardware is optimized to fetch aligned data blocks in a single memory cycle.

If a 4-byte word is stored at a misaligned address (e.g., Address 3 instead of 4), the CPU must perform **two memory access cycles** and shift the bits together, causing a significant performance slowdown.

5. LEVELS OF COMPUTER LANGUAGES

Programs can be authored at three main levels of abstraction:

1. **Machine Language:** The only language directly understood by the computer hardware. Composed entirely of binary digits (0s and 1s). It is machine-dependent.
2. **Assembly Language:** A low-level language that replaces binary code with human-readable textual symbols called **mnemonics** (e.g., ADD, SUB, LOAD). Requires translation.
3. **High-Level Language (HLL):** Highly portable, machine-independent, and human-friendly languages (e.g., C, C++, Java, Python) that use mathematical symbols and English words.

6. PROGRAM TRANSLATORS

Since processors only execute machine code, translators are required:

A. Assembler

Translates assembly language code into native machine language binary instructions.

B. Compiler

Translates high-level language code. It can compile directly into machine language, or first translate into assembly code (which an assembler then converts to machine code).

C. Native Compiler vs. Cross-Compiler

- **Native Compiler/Assembler:** Runs on a specific hardware platform and generates machine code meant to run on that exact same platform.
- **Cross-Compiler/Assembler:** Runs on one computer platform but generates executable machine code for a completely different platform.

Real-World Cross-Compilation Examples:

1. An 8085 cross-assembler running on an Intel desktop PC, generating machine code for an 8085 microprocessor kit.
2. An ARM Embedded-C cross-compiler running on a Windows PC, generating binary code to run on an ARM development board.

Basic Computer Operation & Architecture Types

1. PROGRAMS & SOFTWARE CLASSIFICATION

A **program** or **software** is a structured collection of instructions written to perform a specific computational task or solve a particular problem.

Program Examples:

- Sorting a set of numbers.
- Finding the roots of a quadratic equation.
- Finding the mathematical inverse of a matrix.

Classification of Programs

Programs are broadly divided into two major functional types:

- **Application Software:** Software designed to help the user directly solve a specific real-world, user-level problem.
Examples: Word processors, web browsers, spreadsheet tools, or specialized scientific solvers.
- **System Software:** A collection of background programs that help users build, analyze, manage, and execute application programs.

Key Categories of System Software

1. **Operating System (OS):** Serves as the primary system-level coordinator and resource manager. It bridges the gap between hardware resources and application software.
2. **Compilers & Interpreters:** Translators that convert high-level programming languages (such as C, C++, or Java) into low-level machine instructions.
3. **Supporting Development Tools:** Editors, linkers, debuggers, and assemblers that facilitate software creation.

The Structure of an Operating System (OS)

An Operating System is primarily divided into two main components:

- **Kernel:** The core inner engine of the OS. It runs in privileged mode and directly interacts with the physical hardware. It manages critical tasks such as:
 - Memory management & allocation.
 - CPU scheduling & process execution.
 - File system access & device drivers.
- **Shell:** The outer interface wrapper that surrounds the kernel. It acts as an interpreter that takes user commands and translates them into a format that the kernel can execute.
 - *CLI (Command Line Interface):* Text-based shell (e.g., Command Prompt, Terminal).
 - *GUI (Graphical User Interface):* Visual, icon-based shell (e.g., Windows Desktop, macOS Finder).

2. HARDWARE & INSTRUCTION EXECUTION

The primary components of a computer system are the **CPU (Processor)**, **Memory**, and **I/O (Input/Output) devices**.

To run a program, its low-level machine instructions must be loaded into memory. The processor then reads and executes these instructions one after another.

Essential CPU Registers for Execution

- **Program Counter (PC):** A dedicated control register that holds the memory address of the next instruction waiting to be fetched.
- **Instruction Register (IR):** Holds the binary encoding of the instruction currently being executed by the CPU.

3. THE FETCH-EXECUTE CYCLE

The core lifetime loop of any computer processor is the infinite **Fetch-Execute Cycle**, which operates in 4 continuous steps:

1. **Fetch:** Get instruction from the memory address specified by the PC. Store this instruction inside the IR.
2. **Update:** Increment the PC so it points to the memory address of the next sequential instruction.
3. **Execute:** Decode and perform the operations demanded by the instruction in the IR.
4. **Repeat:** Start the cycle again from Step 1.

4. THE FIVE STAGES OF INSTRUCTION EXECUTION

Modern processors break down the execution of a single instruction into 5 distinct, sequential hardware stages:

1. **Instruction Fetch (IF):** The CPU uses the address stored in the **PC** to retrieve the instruction bytes from memory and stores them in the **IR**.
2. **Instruction Decode (ID):** Control logic decodes the instruction inside the IR to identify the required operation. It simultaneously reads the source operands from the CPU register file.
3. **Execute / ALU Operation (EX):** The Arithmetic Logic Unit (ALU) performs the designated calculation (e.g., addition, subtraction, or logical comparison) using the operands.
4. **Memory Access (MEM):** If the instruction is a memory reference instruction (such as a **LOAD** or **STORE**), the processor reads from or writes data to the RAM.
5. **Write Back (WB):** The calculated result or retrieved memory value is written back to the destination register in the register file.

5. PIPELINING & SYSTEM ARCHITECTURE

Pipelining is an implementation technique where multiple instructions are executed in an overlapped fashion, similar to an assembly line.

Basic 5-Stage Pipelining Visualized

Instr #	C1	C2	C3	C4	C5
Instr 1	IF	ID	EX	MEM	WB
Instr 2	-	IF	ID	EX	MEM
Instr 3	-	-	IF	ID	EX
Instr 4	-	-	-	IF	ID

The Structural Pipeline Bottleneck

Refer to **Clock Cycle 4 (C4)**:

- **Instruction 4** is in its **IF** (Fetch) stage, accessing memory to get its next instruction.
- **Instruction 1** is in its **MEM** stage, accessing memory to read or write data.

Von-Neumann vs. Harvard Architectures

- **Von-Neumann Architecture:** Uses a single unified memory and a shared bus. During C4, a resource conflict (structural hazard) occurs because both stages try to access memory, resulting in a pipeline slowdown.
- **Harvard Architecture:** Uses physically separate memory units and buses for instructions and data. IF and MEM stages can occur simultaneously without any speed penalty.

Instruction Set Architecture (ISA)

1. INTRODUCTION TO ISA

The **Instruction Set Architecture (ISA)** serves as the boundary interface between a computer's software and its physical hardware.

It defines the **programmer's view** of the machine, representing details such as internal registers, data buses, address buses, memory models, and the complete instruction set.

Note: Most ISAs are not restricted to one hardware design; they survive multiple CPU generations (e.g., Intel x86, IBM 360, ARM).

2. KEY DESIGN ISSUES

When developing an ISA, hardware architects must address several critical factors:

- **Explicit Operands:** Can range from 0, 1, 2, to 3 operands.
- **Operand Locations:** Where data is fetched/stored (Registers, Accumulator, Memory, or Stack).
- **Addressing Modes:** Methods used to reference locations (Register, Immediate, Indirect, Relative, etc.).
- **Operand Sizes:** Byte (8-bits), Half-word (16-bits), Word (32-bits), Double-word (64-bits).
- **Operations:** The basic hardware-supported commands (ALU, Data Transfer, Control Flow).

3. ARCHITECTURAL EVOLUTION

Type	Storage Location	Addressing
Accumulator	Accumulator (ACC)	1 implicit, 1 explicit
Stack	LIFO Stack (RAM)	0 address (implicit)
Mem-Mem	Main Memory (RAM)	2 or 3 explicit memory
Reg-Mem	Registers & Memory	1 Register, 1 Memory
Load-Store	GPRs only (MIPS/ARM)	3 Registers (ALU only)

4. CPU ARCHITECTURES VS. MATH CODE

How different designs execute calculations for $Y = X + Y$:

A. Stack Architecture (0-address)

Pops operands from the top of the stack, executes math, and pushes results back.

```
PUSH X      ; Stack: [X]
PUSH Y      ; Stack: [Y, X]
ADD         ; Stack: [(X+Y)]
POP Z       ; Z = X+Y
```

B. Accumulator Architecture (1-address)

Implicitly uses the ACC register for one of the operands and the target output.

```
LOAD X      ; ACC = X
ADD Y       ; ACC = ACC + Y
STORE Z     ; Z = ACC
```

C. Load-Store Architecture (Register-Register)

Math is strict to registers. Memory access occurs only via LOAD and STORE.

```
LOAD R1, X  ; R1 = Mem[X]
LOAD R2, Y  ; R2 = Mem[Y]
ADD R3, R1, R2 ; R3 = R1 + R2
STORE Z, R3 ; Mem[Z] = R3
```

5. EVALUATION OF COMPLEX MATH

Evaluating: $Y = \frac{A}{B} - (A - C \times B)$

Stack-Based Execution

```
PUSH A
PUSH B
DIV      ; Stack: [A/B]
PUSH A
PUSH C
PUSH B
MUL      ; Stack: [C*B, A, A/B]
SUB      ; Stack: [A-(C*B), A/B]
SUB      ; Stack: [(A/B) - (A-(C*B))]
POP Y    ; Output to Y
```

Accumulator-Based Execution

```
LOAD C
MUL B
STORE T1 ; T1 = C * B
LOAD A
SUB T1
STORE T1 ; T1 = A - (C * B)
LOAD A
DIV B     ; ACC = A / B
SUB T1    ; ACC = (A / B) - T1
STORE Y   ; Y = ACC
```

Memory-to-Memory (3-Address)

```
DIV T1, A, B ; T1 = A / B
MUL T2, C, B ; T2 = C * B
SUB T2, A, T2 ; T2 = A - T2
SUB Y, T1, T2 ; Y = T1 - T2
```

Load-Store Execution (Modern standard)

```
LOAD R1, A
LOAD R2, B
LOAD R3, C
DIV R4, R1, R2 ; R4 = A / B
MUL R5, R3, R2 ; R5 = C * B
SUB R5, R1, R5 ; R5 = A - R5
SUB R4, R4, R5 ; R4 = R4 - R5
STORE Y, R4    ; Save result to Y
```

6. GENERAL PURPOSE REGISTERS (GPR)

Modern computing relies entirely on General Purpose Registers (GPRs). Instead of dedicating registers for single tasks, any GPR can store data temporarily.

Key Advantages of GPR:

- **High Speed:** Registers operate directly within the CPU core, running orders of magnitude faster than cache or RAM.
- **Compiler Optimization:** Compilers can easily allocate registers to active variables.
- **Less RAM Traffic:** Stores temporary loop and calculation data, preventing bus congestion.

The Major Trade-off:

During context switches or interrupt processing, the OS must save the states of all GPRs into memory. Having a massive register file increases context-switching overhead.

Basic Number Systems

1. INTRODUCTION TO NUMBER SYSTEMS

In computer architectures, numeric data is structured using different positional number systems. Every digit position carries a specific weight determined by its **base (radix)**.

A. Decimal System (Radix = 10)

Uses 10 distinct digits: 0 to 9. Weights are powers of 10.

Examples:

- $234 = 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$
- $250.67 = 2 \times 10^2 + 5 \times 10^1 + 0 \times 10^0 + 6 \times 10^{-1} + 7 \times 10^{-2}$

B. Binary System (Radix = 2)

Uses 2 digits: 0 and 1. This is the native language of computer hardware.

Example:

- $110.11_2 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$
 $= 4 + 2 + 0 + 0.5 + 0.25 = 6.75_{10}$

C. Octal System (Radix = 8)

Uses 8 digits: 0 to 7. Highly useful for compacting binary patterns.

Example:

- $237.4_8 = 2 \times 8^2 + 3 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1}$

D. Hexadecimal System (Radix = 16)

Uses 16 characters: 0-9 and A-F (where A=10, B=11, C=12, D=13, E=14, F=15).

Example:

- $BA.4_{16} = 11 \times 16^1 + 10 \times 16^0 + 4 \times 16^{-1}$

2. BASE CONVERSIONS

A. Any Radix to Decimal

Multiply each digit by its positional weight and sum them up.

Convert 1011.01_2 to Decimal:

- $= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2})$
 $= 8 + 0 + 2 + 1 + 0 + 0.25 = 11.25_{10}$

B. Decimal to Binary (Integer Part)

Use **repeated division by 2**. Keep track of remainders from bottom to top (MSB to LSB).

Convert 25_{10} to Binary:

- $25 / 2 = 12 \text{ r } 1 \text{ (LSB)}$
- $12 / 2 = 6 \text{ r } 0$
- $6 / 2 = 3 \text{ r } 0$
- $3 / 2 = 1 \text{ r } 1$
- $1 / 2 = 0 \text{ r } 1 \text{ (MSB)}$ => Result: 11001_2

C. Decimal to Binary (Fractional Part)

Use **repeated multiplication by 2**. The integer parts of the products form the binary fraction from top to bottom.

Convert 0.625_{10} to Binary:

- $0.625 \times 2 = 1.250 \rightarrow \text{Take } 1$
- $0.250 \times 2 = 0.500 \rightarrow \text{Take } 0$
- $0.500 \times 2 = 1.000 \rightarrow \text{Take } 1 \text{ (Stop at } 0) \Rightarrow \text{Result: } .101_2$

D. Hex / Octal <--> Binary Conversions

These conversions are highly direct because $16 = 2^4$ and $8 = 2^3$.

- Binary to Octal:** Group binary bits into sets of 3 (starting from the radix point) and convert.
- Binary to Hex:** Group binary bits into sets of 4 and convert.

Convert 11110.11_2 to Hex:

- Group by 4: $0001 \ 1110 \ . \ 1100_2$ (padded with zeros)
- Hex value: $1 \ E \ . \ C_{16} \Rightarrow 1E.C_{16}$

3. REPRESENTING SIGNED NUMBERS

Computers must represent positive and negative values. There are 3 main representations:

A. Sign-and-Magnitude

The Most Significant Bit (MSB) acts as the sign bit (0 = positive, 1 = negative). The remaining bits show the absolute magnitude.

Drawbacks: Has two representations for zero (+0 and -0). Hardware implementation of addition/subtraction is complex.

B. 1's Complement

To represent a negative number, invert all bits of its positive equivalent.

Drawback: Still contains two representations for zero (+0 = 0000, -0 = 1111 in a 4-bit system).

4. DEEP DIVE: 2'S COMPLEMENT

To form a negative number in **2's Complement**, find its 1's complement (flip all bits) and then add 1.

Represent -5_{10} in 4-bit 2's Complement:
 $+5$ in Binary = 0101_2
 1's Complement = 1010_2 (Flip bits)
 Add 1 = 1_2
 Result (-5_{10}) = 1011_2

Why 2's Complement Rules Modern Design:

- Unique Zero:** Only one representation of zero (e.g., in 4-bit: 0000₂). Attempting to negate 0000₂ yields 0000₂.
- Unified Circuitry:** Subtraction can be performed directly using addition hardware: $\$A - \$B = \$A + (-\$B)\$$. This simplifies logic circuitry significantly.

Value Formula (Weighted Representation)

The MSB in a 2's complement number has a negative weight of $\$-2^{(n-1)}\$$:

$$D = -b_{(n-1)} \cdot 2^{(n-1)} + b_{(n-2)} \cdot 2^{(n-2)} + \dots + b_0 \cdot 2^0$$

Verify 1011_2 (4-bit 2's Complement):

- Weight of bits: $-2^3 \mid 2^2 \mid 2^1 \mid 2^0$
- Calculation: $-8 \times 1 + 4 \times 0 + 2 \times 1 + 1 \times 1 = -8 + 3 = -5_{10}$

5. ESSENTIAL PROPERTIES & OPERATIONS

A. Representable Ranges (for n bits)

System	Minimum Value	Maximum Value
Sign-Mag	$\$-(2^{(n-1)} - 1)\$$	$\$(2^{(n-1)} - 1)\$$
1's Comp	$\$-(2^{(n-1)} - 1)\$$	$\$(2^{(n-1)} - 1)\$$
2's Comp	$\$-2^{(n-1)}\$$	$\$(2^{(n-1)} - 1)\$$

Example Ranges for $n = 4$ bits:

- Sign-Magnitude: -7 to +7
- 2's Complement: -8 to +7 (Min: $-2^3 = -8$, Max: $2^3 - 1 = 7$)

B. Arithmetic Shifting

- Arithmetic Left Shift (by k bits):** Shift all bits to the left and pad with 0s. This multiplies the number by $\$2^k\$$.
- Arithmetic Right Shift (by k bits):** Shift all bits to the right and pad with the **sign bit**. This divides the number by $\$2^k\$$.

Right Shift 1010_2 (-6_{10}) by 1 bit:

Pad with Sign Bit (1) from left: 1101_2 ($= -3_{10}$)

Left Shift 0011_2 (3_{10}) by 1 bit:

Pad with 0 from right: 0110_2 ($= 6_{10}$)

C. Sign Extension

To increase the bit size of a 2's complement number (e.g., from 4-bit to 8-bit), copy the sign bit into the new upper positions. The numeric value remains unchanged.

Examples:

- +5 (4-bit): $0101_2 \Rightarrow$ (8-bit): 00000101_2
- 5 (4-bit): $1011_2 \Rightarrow$ (8-bit): 11111011_2

Note: Almost all modern microprocessors use 2's Complement representation for representing signed integer data.

Instruction Formats & Addressing Modes

1. ANATOMY OF AN INSTRUCTION

An instruction is a binary machine command that tells the processor what action to perform. It consists of two essential parts:

- **Operation Code (Opcode):** Specifies the hardware task to be executed.
Categories: Data transfer (LOAD, STORE, MOV), Arithmetic/Logical (ADD, AND), Control (JMP, BR), I/O, and system control.
- **Operand(s):** Identifies the source data values or destination locations for the operation.
Sources: Constant values (Immediate), Registers, or Memory Addresses.

Concept of Effective Address (EA)

The **Effective Address (EA)** is the actual physical address of a memory location containing the target operand, calculated dynamically by the CPU during instruction execution.

2. COMPLETE GUIDE TO ADDRESSING MODES

Addressing modes define how the CPU determines the location of operands. Different modes optimize execution speed, memory footprint, or array access:

A. Immediate Addressing

The operand value is written directly within the instruction itself. No memory reference is needed.

Syntax: ADD R1, #150 (or MOV R2, 200)
Effective Address (EA): Not applicable (Value is inside instruction)
Pros: Extremely fast (no memory cycles).
Cons: Limited range for constant sizes.

B. Register Addressing

The operand is located inside a specified CPU register.

Syntax: ADD R1, R2
Effective Address (EA): Operand is in Register R2
Pros: Very fast; registers reside within the CPU core clock.
Cons: Limited number of CPU registers available.

C. Direct (Absolute) Addressing

The instruction contains the exact physical memory address of the operand.

Syntax: LOAD R1, 1000
Effective Address (EA): EA = 1000
How it works: CPU looks up Memory location 1000 to get data.
Pros: Straightforward memory access.
Cons: Address space is limited by instruction word size.

D. Indirect Addressing

The instruction points to a memory location (or register) that contains the *address* of the actual operand.

Syntax: LOAD R1, (1000)
Effective Address (EA): EA = Memory[1000]
How it works: If Memory[1000] contains '2050', the CPU retrieves the data from address '2050'.
Pros: Allows dynamic pointer references.
Cons: Requires two memory accesses, slowing performance.

E. Register Indirect Addressing

The address of the operand is held inside a CPU register.

Syntax: LOAD R1, (R2)
Effective Address (EA): EA = [R2]
Pros: Dynamic addressing; faster than standard memory-indirect because the pointer is in a register.

F. Relative Addressing

The Effective Address is calculated by adding an offset/displacement directly to the **Program Counter (PC)**.

Formula: $EA = [PC] + \text{Offset}$
Usage: Essential for local branching/jumping statements (e.g., JUMP +8).

G. Indexed Addressing

The Effective Address is computed by adding a constant offset displacement to the contents of a specified Index Register (\$R_i\$).

Formula: $EA = [R_i] + \text{Offset}$
Usage: Ideal for stepping through arrays. The base index of the array is the Offset, and \$R_i\$ acts as the loop index pointer.

H. Autoincrement Addressing

The address of the operand is in a register. After accessing the operand, the value in that register is automatically incremented by the size of the data type.

Syntax: LOAD R1, (R2)+
Step 1: Fetch operand at EA = [R2]
Step 2: Update register: $R2 = R2 + d$ (where d is data size)

I. Autodecrement Addressing

The value in the register is decremented first, and then this new value is used as the Effective Address.

Syntax: LOAD R1, -(R2)
Step 1: Update register: $R2 = R2 - d$
Step 2: Fetch operand at EA = [R2]

3. CPU REGISTER CONFIGURATIONS & ROLES

Modern RISC/GPR architectures assign specific operational tasks to individual registers to manage programs efficiently:

A. Pointer Registers

- **Global Area Pointer (GP):** Points to the base address in memory where global and static variables are permanently allocated.
- **Stack Pointer (SP):** Points to the top-most active memory address of the system stack. Handles push/pop during execution.
- **Frame Pointer (FP):** Points to the base activation record (stack frame) of the currently active function, managing local variables.
- **Return Address (RA):** Holds the memory address to JUMP back to after a function call (subroutine) completes.

B. Parameter Passing & Kernel Registers

- **Argument Registers:** Typically, up to 4 GPRs are dedicated to passing function arguments directly, avoiding slow stack memory writes.
- **Kernel Registers:** Registers strictly reserved for use by the Operating System Kernel. General applications must never write to these registers to prevent crashes or security breaches.

4. SELF-ASSESSMENT TRACING EXERCISES

Scenario: Consider a system with the following state:

- Register **R1** = 2000
- Register **R2** = 3000
- Memory location **2000** = 50
- Memory location **3000** = 2000
- Memory location **2004** = 90

Determine the Effective Address (EA) and retrieved data value for the following instructions:

Instruction	Addressing Mode	EA	Value
LOAD R3, 2000	Direct (Absolute)	2000	50
LOAD R3, (R1)	Register Indirect	2000	50
LOAD R3, (R2)	Indirect (Memory)	2000	50
LOAD R3, 4(R1)	Indexed (Offset = 4)	2004	90
LOAD R3, (R1)+	Autoincrement	2000	50 (R1 becomes 2004)

Q1. Explain why Autoincrement/Autodecrement modes are highly beneficial in loop structures.

Answer: Without these modes, stepping through an array requires two instructions: one to fetch the data and another to increment the loop pointer register (e.g., 'ADD R1, #4'). Autoincrement merges these into one instruction cycle, decreasing program size and improving loop execution speed.

MIPS32 Instruction Set Architecture

1. MIPS32 DESIGN OVERVIEW

MIPS32 is a classic high-performance 32-bit RISC (Reduced Instruction Set Computer) architecture.

Core Architectural Rules:

- **Fixed-Width Instructions:** Every single instruction is exactly 32 bits long.
- **Load-Store Design:** Arithmetic/logical computations can only operate on registers. Accessing memory is strictly limited to `lw` (load word) and `sw` (store word) instructions.
- **Memory Word Alignment:** Every word (32 bits / 4 bytes) must start at a memory address that is a multiple of 4 (i.e., addresses ending in binary `00` like `0x0000`, `0x0004`, `0x0008`). This ensures a word can be fetched in a single memory cycle.

2. MIPS32 REGISTER FILE (32 GPRS)

MIPS provides 32 general-purpose registers (GPRs), each 32 bits wide. By convention, they are named and used as follows:

Reg Name	Number	Usage & Purpose
\$zero	0	Hardwired to constant 0 (read-only)
\$at	1	Reserved for Assembler Temporary
\$v0 - \$v1	2 - 3	Values returned by functions
\$a0 - \$a3	4 - 7	Arguments passed to functions
\$t0 - \$t7	8 - 15	Temporary variables (not saved across calls)
\$s0 - \$s7	16 - 23	Saved variables (preserved across calls)
\$t8 - \$t9	24 - 25	More temporary variables
\$k0 - \$k1	26 - 27	Reserved strictly for OS Kernel
\$gp	28	Global Pointer (static memory base)
\$sp	29	Stack Pointer (active stack top)
\$fp	30	Frame Pointer (activation record base)
\$ra	31	Return Address (subroutine JAL target)

3. MIPS32 INSTRUCTION FORMATS

All instructions fall into one of three structural layout formats:

A. R-Type (Register Format)

Used for ALU operations containing three register operands.

6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
opcode	rs	rt	rd	shamt	funct
• opcode (31:26): Operation code. For R-type, this is always 0.					
• rs (25:21): 1st source register.					
• rt (20:16): 2nd source register.					
• rd (15:11): Destination register.					
• shamt (10:6): Shift amount (used for <code>sll</code> , <code>srl</code> ; else 0).					
• funct (5:0): Function code (selects specific ALU operation).					

Example: `add $t0, $s1, $s2`

B. I-Type (Immediate/Offset Format)

Used for operations with constants, load/store, and branching.

6 bits	5 bits	5 bits	16 bits
opcode	rs	rt	immediate / offset
• rs (25:21): Source register.			
• rt (20:16): Target register (source for <code>sw</code> , destination for <code>lw</code> / <code>addi</code>).			
• immediate (15:0): 16-bit constant/offset (-32768 to +32767).			

Example: `addi $s1, $s2, 200` or `lw $t0, 40($s3)`

C. J-Type (Jump Format)

Used for unconditional jump instructions.

6 bits	26 bits
opcode	target_address
• target_address (25:0): Base jump address.	

Example: `j Label`

4. ADDRESSING MODES IN MIPS32

MIPS32 utilizes five highly optimized addressing modes:

- **Register Addressing:** The operand is located inside a register.
`add $s1, $s2, $s3`
- **Immediate Addressing:** The operand is a 16-bit constant.
`addi $s1, $s2, 200`
- **Base (Displacement) Addressing:** The operand's Effective Address is calculated by adding a register's content to a 16-bit sign-extended displacement.
`lw $s1, 150($s2)  $\rightarrow$  $EA = [$s2] + 150$`
- **PC-Relative Addressing:** Used for branching. A 16-bit offset is shifted left by 2 bits (multiplied by 4 to align with instructions) and added to the updated `$PC+4$`.
`beq $s1, $s2, Label  $\rightarrow$  $Target = [PC+4] + (Offset  $\times$  4)$`
- **Pseudo-Direct Addressing:** Used for jumps. The 26-bit immediate field is shifted left by 2 bits (making 28 bits) and combined with the upper 4 bits of the updated `$PC+4$` to form a 32-bit address.
`j Label  $\rightarrow$  $Target = PC_{[31..28]} \mid (TargetAddress \times 4)$`

5. CRITICAL EXAM-STYLE CODE EXERCISES

Exercise A: Binary Encoding (Assembly to Machine)

Question: Convert the MIPS instruction `add $t0, $s1, $s2` into its 32-bit hexadecimal machine code.

Reference Data:

- `add` is an R-Type instruction.
- Opcode for R-type = `000000`. Function code for `add` = `100000` (`$32_{10}`).
- Register numbers: `$t0` = 8 (`01000`), `$s1` = 17 (`10001`), `$s2` = 18 (`10010`).
- Shift amount (`shamt`) = 0 (`00000`).

Mapping to Fields:

Field	op	rs	rt	rd	shamt	funct
Decimal	0	17	18	8	0	32
Binary	000000	10001	10010	01000	00000	100000

Combine fields: `0000 0010 0011 0010 0100 0000 0010 0000`

Convert to Hexadecimal: **0x02324020**

Exercise B: Base Offset Decoding

Question: Suppose register `$s2` holds the value `0x10002000`. Calculate the effective address for `lw $s1, 16($s2)` and explain how the CPU loads the data.

Answer:

- Base register content = `0x10002000`.
- Offset displacement = 16 (Decimal) = `0x00000010` (Hex).
- Effective Address calculation: $\text{\$EA} = [\text{\$s2}] + \text{\text{offset}} = \text{\text{0x10002000}} + \text{\text{0x00000010}} = \text{\text{0x10002010}}$
- The CPU fetches a 32-bit word from memory starting at `0x10002010` and stores it inside register `$s1`. Note that the address `0x10002010` is perfectly aligned because its last digit is 0, which is divisible by 4.

Exercise C: Branch Target Offset Calculation

Question: During compilation, a branch instruction `beq $s0, $s1, Target` is located at memory address `0x00400010`. The compiler sets the 16-bit offset field inside the machine instruction to `0x000A` (10 in decimal). What is the exact target address of the branch?

Answer:

- Current address of instruction = `0x00400010`.
- Updated `$PC+4$` = `0x00400010` + 4 = `0x00400014`.
- Shift offset left by 2 (multiplied by 4): `10  $\times$  4 = 40$ bytes = 0x00000028` (Hex).
- Target Address calculation: $\text{\$Target} = [\text{\$PC+4}] + (\text{\text{Offset}} \times 4) = \text{\text{0x00400014}} + \text{\text{0x00000028}} = \text{\text{0x0040003C}}$

Design of Control Unit

1. INSTRUCTION FETCH & MEMORY ADDRESSING

Instructions are permanently stored in the primary memory (RAM). The execution sequence is managed by the **Program Counter (PC)**.

- **Instruction Length:** In MIPS32, every instruction is exactly 4 bytes (32 bits) long.
- **Byte Addressability:** Since memory is byte-addressable, instructions must start at addresses that are multiples of 4 (e.g., addresses ending in binary 00).
- **Sequential Execution:** Unless a branch or jump occurs, the PC is incremented by 4 ($\$PC \leftarrow PC + 4$) during every fetch cycle to point to the next instruction word.

2. EXECUTION OF A COMPLETE INSTRUCTION

A single machine instruction is executed in sequence using clock cycles denoted as **Time Slots (\$T_1, T_2, T_3...\$)**. Let's look at the exact control signals for different operations in a **Single-Bus Organization**:

A. Instruction Fetch Phase (Common to ALL)

Every instruction must first be fetched and decoded. This always takes slots $\$T_1$ to $\$T_3$:

```
T1: PC_out, MAR_in, Read, Select4, Add, Z_in
    ; Move PC address to MAR, start memory Read,
    ; ALU computes PC + 4, stores result in Z.
T2: Z_out, PC_in, Y_in, WMFC
    ; Update PC from Z, wait for memory data (WMFC).
T3: MDR_out, IR_in
    ; Place fetched instruction on bus, load to IR.
```

B. Arithmetic Addition: ADD R1, R2

This instruction performs $\$R_1 \leftarrow R_1 + R_2$ using the fetched instruction in the IR ($\$T_4$ to $\$T_6$):

```
T4: R1_out, Y_in
    ; Load the first operand R1 into temporary register Y.
T5: R2_out, SelectY, Add, Z_in
    ; Place R2 on bus, select Y, add them, store to Z.
T6: Z_out, R1_in, End
    ; Save the sum from Z back to destination R1, End.
```

C. Load Word: MOVE R1, (R2)

Fetches a data word from RAM at the memory address pointed to by register $\$R_2$ and loads it into $\$R_1$:

```
T4: R2_out, MAR_in, Read
    ; Move memory pointer in R2 to MAR, start RAM Read.
T5: MDR_inE, WMFC
    ; Wait for RAM to place data on bus, load to MDR.
T6: MDR_out, R1_in, End
    ; Transfer retrieved data from MDR to register R1.
```

D. Store Word: MOVE (R1), R2

Writes the data in register $\$R_2$ into the memory address pointed to by register $\$R_1$:

```
T4: R1_out, MAR_in
    ; Move memory pointer in R1 to MAR.
T5: R2_out, MDR_in, Write
    ; Load register R2 data into MDR, initiate RAM Write.
T6: WMFC, End
    ; Wait for memory write function to complete, End.
```

3. THREE-BUS ORGANIZATION OPTIMIZATION

A **Three-Bus Organization** has three independent internal buses (Bus A, Bus B, Bus C). This completely eliminates temporary registers Y and Z, allowing execution in fewer steps.

Example: SUB R1, R2, R3 ($\$R_1 \leftarrow R_2 - R_3$)

```
T1: PC_out, R=B, MAR_in, Read, IncPC
    ; PC sent on Bus B, directly copied to MAR,
    ; PC is incremented directly via hardware (IncPC).
T2: WMFC
    ; Wait for memory function to complete.
T3: MDR_outB, IR_in
    ; Transfer fetched instruction to IR.
T4: R2_outA, R3_outB, Sub, R1_in, End
    ; Source 1 on Bus A, Source 2 on Bus B.
    ; ALU subtracts them, writes back to R1 via Bus C.
```

Note: Arithmetic subtraction is completed in only **one cycle** ($\$T_4$) instead of three!

4. CONTROL UNIT IMPLEMENTATIONS

There are two primary methods to design the hardware of a Control Unit (CU):

A. Hardwired Control Unit

Uses dedicated combinational logic circuits, decoders, and step counters to generate signals directly from hardware.

Inputs (IR, Flags, Clock) $\rightarrow$ [Decoder / Logic Gates] $\rightarrow$ Control Signals

- **Inputs:** Instruction Register (IR), Condition Codes/Flags, Clock/Step Counter.
- **Pros:** Extremely fast (operates at native speed of logic gates). Highly optimized.
- **Cons:** Inflexible. Any modification to the instruction set requires physically redesigning the microchip's wiring.

B. Microprogrammed Control Unit

Control signals are stored as binary words (Microinstructions) in a dedicated fast, read-only memory inside the CPU called the **Control Store (ROM)**.

[MicroPC] $\rightarrow$ [Control Store (ROM)] $\rightarrow$ [Micro-Instruction Register] $\rightarrow$ Control Signals
 $\uparrow$
 [Next Address Generator]

- **How it works:** The instruction in the IR points to the starting address of a microprogram. The **Micro-PC (μPC)** steps through the Control Store, loading the micro-instructions which directly activate the control lines.
- **Pros:** Highly flexible. Complex instructions can be added or updated simply by rewriting the micro-code in ROM.
- **Cons:** Slower because every control step requires a lookup cycle in the Control Store ROM.

5. HIGH-YIELD PRACTICE QUESTIONS

Q1. Why are temporary registers Y and Z not needed in a Three-Bus Organization?

Answer: In a single-bus system, only one operand can occupy the bus at a time, requiring Y to hold one input and Z to hold the output. A three-bus system has separate buses (A and B) to feed both ALU inputs simultaneously, and bus C to route the output directly to the destination register, completing the operation in a single clock cycle.

Q2. Write down the control sequence for executing the MIPS instruction lw \$t0, 8(\$t1) in a Single-Bus Organization (assuming fetch is complete).

Answer:

```
T4: T1_out, Y_in
    ; Load base address from register $t1 into Y.
T5: Offset_out, SelectY, Add, Z_in
    ; Add constant Offset (8) to Y, store EA in Z.
T6: Z_out, MAR_in, Read
    ; Move EA to MAR, start memory read.
T7: MDR_inE, WMFC
    ; Wait for memory, load data to MDR.
T8: MDR_out, T0_in, End
    ; Move retrieved word to register $t0, End.
```

Memory Systems Architecture

1. PROCESSOR-MEMORY CONNECTION

Memory is the vital functional unit used to store program instructions and data. The communication interface between the CPU and Primary Memory consists of three buses:

- **Address Bus:** Unidirectional. Provides the memory address location the CPU wants to access.
- **Data Bus:** Bidirectional. Transfers data from memory to CPU (Read) or from CPU to memory (Write).
- **Control Bus:** Coordinates timing and commands by carrying control signals (e.g., READ, WRITE, MFC).

Fundamental Memory Definitions

- **Byte-Addressable Memory:** Each individual byte (8 bits) has a unique address. This is the modern industry standard.
- **Word-Addressable Memory:** Each unique address points to a complete word (multiple bytes, e.g., 32 bits or 4 bytes).

2. HARDWARE STEPS OF READ/WRITE CYCLES

The interactions during memory transactions are highly structured:

A. Memory Read Operation Sequence

1. The CPU loads the target address into the **Memory Address Register (MAR)**.
2. The address is placed on the Address Bus. The Control Unit asserts the **READ** control signal.
3. The memory subsystem decodes the address, retrieves the data, and places it on the Data Bus.
4. Memory asserts the **Memory Function Complete (MFC)** signal to let the CPU know the data is ready.
5. The CPU reads the data from the Data Bus and stores it in the **Memory Data Register (MDR)**.

B. Memory Write Operation Sequence

1. The CPU loads the target write address into the **MAR**.
2. The CPU loads the data to be written into the **MDR**.
3. The address is placed on the Address Bus, and data is placed on the Data Bus.
4. The Control Unit asserts the **WRITE** signal.
5. The memory subsystem writes the data into the selected address and returns the **MFC** signal once complete.

3. PRINCIPLE OF MEMORY HIERARCHY

A single type of memory cannot be fast, cheap, and massive at the same time. This is known as the Memory Design Dilemma.

The Solution: Arrange memories hierarchically. Smaller, fast, expensive memory (Cache) sits near the CPU, backed by larger, slower, cheaper memory (RAM, SSD).

Locality of Reference

The memory hierarchy works effectively because programs exhibit **Locality of Reference**. Over a short time frame, the CPU accesses only a small fraction of the total memory space:

- **Temporal Locality (Locality in Time):** If a memory location is accessed once, it is highly likely to be accessed again in the near future (e.g., variables in loops like `i++`).
- **Spatial Locality (Locality in Space):** If a memory location is accessed, nearby locations are highly likely to be accessed soon (e.g., consecutive array elements or sequential instructions).

4. PERFORMANCE METRICS & FORMULAS

The following metrics assess the performance of a multi-level memory hierarchy (typically L1 Cache and L2 Main Memory):

A. Fundamental Variables

- **\$H\$ (Hit Rate):** The fraction of memory accesses successfully found in the upper level (Cache). Value range is $0 \leq H \leq 1$.
- **\$1 - H\$ (Miss Rate):** The fraction of memory accesses not found in the upper level, requiring a lookup in lower memory.
- **\$t_{A1}\$ (Hit Time):** Time required to access the upper level (includes cache read time and hit/miss determination).
- **\$t_{A2}\$ (Miss Penalty):** The total time overhead when a cache miss occurs (time to access lower level + block transfer to upper level).

B. Mathematical Equations

1. Average Access Time (\$t_A\$):

The average time the CPU spends per memory access:

$$t_A = H \cdot t_{A1} + (1 - H) \cdot t_{A2}$$

2. Access Efficiency (\$e\$):

Indicates how close the average access time is to the ideal minimum access time (\$t_{A1}\$):

$$e = t_{A1} / t_A = 1 / [H + (1 - H) \cdot r]$$

Where $r = t_{A2} / t_{A1}$ is the speed ratio between the lower and upper memory levels.

3. Speedup (\$S\$):

The performance gain achieved by using the hierarchy compared to accessing the slower level (\$t_{A2}\$) directly:

$$S = t_{A2} / t_A = 1 / [(H / r) + (1 - H)]$$

Note: This equation directly aligns with Amdahl's Law.

5. WORKED MATH TRACING EXERCISES

Sample Problem:

A computer system has an L1 Cache with an access time (\$t_{A1}\$) of **2 ns** and a Main Memory with an access time of **50 ns**. If a cache miss occurs, the total miss penalty (\$t_{A2}\$) is **52 ns**. The system achieves a Hit Rate (\$H\$) of **0.95 (95%)**.

Q1. Calculate the Average Access Time (\$t_A\$).

Calculation: $t_A = H \cdot t_{A1} + (1 - H) \cdot t_{A2}$
 $t_A = (0.95 \cdot 2) + (1 - 0.95) \cdot 52$
 $t_A = 1.9 + (0.05 \cdot 52) = 1.9 + 2.6 = 4.5 \text{ ns}$

Q2. Calculate the Access Efficiency (\$e\$) of this system.

Calculation: $e = t_{A1} / t_A = 2 \text{ ns} / 4.5 \text{ ns}$
 $\approx 0.444 \text{ (or } 44.4\%)$

Q3. Determine the Speedup (\$S\$) gained by adding this Cache.

Calculation: $S = t_{A2} / t_A = 52 \text{ ns} / 4.5 \text{ ns}$
 $\approx 11.56 \text{ times faster}$

Input-Output Systems Interfacing

1. THE I/O INTERFACING CHALLENGE

Interfacing peripheral devices (I/O) to a computer is far more complex than interfacing memory subsystems.

Why is I/O Interfacing Complex?

- **Wide Variety:** Peripherals use highly diverse mechanical and electronic operating principles (keyboard, mouse, disk, printer).
- **Speed Mismatch:** I/O devices are usually several orders of magnitude slower than the CPU and RAM.
- **Data Rates:** Data transfers can be extremely regular (video streaming) or completely irregular (keyboard keypresses).
- **Data Formats:** Word lengths and data block sizes vary widely (bytes vs. kilobytes).

The Solution: The I/O Module

To bridge this gap, we use a programmable **I/O Module** (or I/O Interface). It connects to the CPU/Memory bus on one side and controls the peripherals on the other side.

Inside an I/O Module Interface:

- **Data Register:** Holds data being transferred to or from the peripheral.
- **Status Register:** Holds flags indicating the state of the device (e.g., Busy, Ready, Error).
- **Control Register:** Receives commands from the CPU to configure or operate the peripheral.

2. I/O ADDRESSING SCHEMES

The CPU must have a way to address and communicate with these I/O registers. There are two primary schemes:

A. Memory-Mapped I/O

The I/O registers are assigned addresses within the normal **unified memory address space**.

- **How it works:** The CPU uses standard memory access instructions (like LOAD, STORE, lw, sw) to communicate with I/O devices.
- **Pros:** No special CPU instructions needed; highly flexible programming.
- **Cons:** Uses up valuable memory address space.

B. Isolated I/O (I/O-Mapped I/O)

The I/O devices have a **completely separate address space**, separate from primary memory.

- **How it works:** The CPU must use dedicated instructions (like IN and OUT) to access I/O ports. It utilizes an extra control line on the bus to switch between Memory and I/O access.
- **Pros:** Preserves the full memory address range for RAM.
- **Cons:** Limited instruction set; more complex CPU control logic.

3. THREE DATA TRANSFER METHODS

How data is physically moved between the CPU/Memory and I/O:

1. Programmed I/O (Polling)

The CPU is in complete control. It continuously polls (loops and checks) the status register of the I/O module to see if it is ready.

- **Drawback:** Highly inefficient. The CPU is trapped in a useless wait loop (polling), wasting billions of clock cycles.

2. Interrupt-Driven I/O

The CPU issues an I/O command and goes back to executing other background programs. When the I/O device becomes ready, it alerts the CPU by asserting an **Interrupt Request (IRQ)** line.

- **Benefit:** No wasted CPU polling cycles.

3. Direct Memory Access (DMA)

A specialized hardware unit (the DMA Controller) takes control of the system bus to transfer large blocks of data directly between high-speed peripherals and RAM without CPU intervention.

4. INTERRUPT HANDLING & PRIORITY

When a CPU receives an interrupt, it completes the current instruction, saves the program state (PC, Registers) to the stack, and JUMPs to an **Interrupt Service Routine (ISR)**.

Handling Multiple Interrupt Requests

When multiple devices request attention, the CPU must prioritize them. Two main techniques are used:

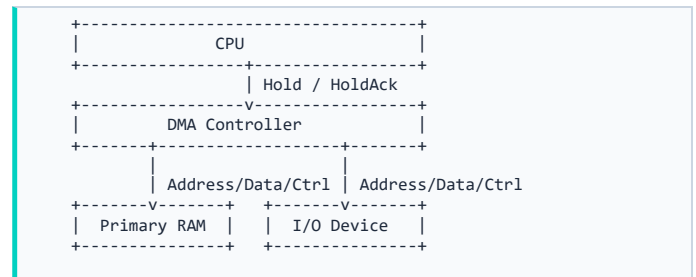
- **Daisy Chaining (Hardware Priority):** Devices are connected in a physical serial chain. The interrupt acknowledge signal (INTA) propagates through the devices. The device closest to the CPU has the highest priority and intercepts the signal.
 - *Pro:* Simple hardware. *Con:* Inflexible; slow propagation delay.
- **Vectored Interrupts:** The interrupting device directly sends a unique binary index code (Interrupt Vector) to the CPU. The CPU uses this vector to look up the starting address of the corresponding ISR in an **Interrupt Vector Table (IVT)**, allowing instant execution without polling.

5. DEEP-DIVE: DIRECT MEMORY ACCESS (DMA)

For high-speed devices (like hard drives or network cards), interrupting the CPU for every byte is too slow.

The **DMA Controller (DMAC)** acts as a secondary processor. It contains three main registers:

- **Address Register:** Holds the starting RAM address for transfer.
- **Word Count Register:** Holds the number of data words to transfer (decrements after each transfer).
- **Control/Status Register:** Manages transfer settings and reports completion.



DMA Transfer Modes

How the DMAC shares the system bus with the CPU:

1. **Cycle Stealing:** The DMAC requests the bus, transfers a single data word, and immediately releases the bus back to the CPU. It "steals" clock cycles without the CPU noticing, allowing both to run in parallel.
2. **DMA Block Transfer (Burst Mode):** The DMAC takes complete control of the bus and transfers an entire block of data continuously. The CPU must pause (lie idle) because it cannot access RAM to fetch instructions during this burst.

Other Key Applications of DMA:

- **Memory-to-Memory Moves:** Extremely fast block transfers of data inside RAM.
- **Dynamic RAM Refreshing:** Periodically generating dummy read requests to columns in DRAM to prevent data loss.

6. HIGH-YIELD PRACTICE QUESTIONS

Q1. Compare and contrast Memory-Mapped I/O and Isolated I/O.

Feature	Memory-Mapped I/O	Isolated I/O
Address Space	Shared with memory.	Completely separate.
Instructions	Standard (lw, sw).	Specialized (IN, OUT).
Control Signals	Simple read/write.	Requires an I/O select line.

Q2. Why is Programmed I/O considered highly inefficient for slower peripherals like keyboards?

Answer: Keyboards are extremely slow. In Programmed I/O, the CPU must sit in a continuous loop reading the keyboard status register over and over until a key is pressed. This blocks the CPU from doing any other useful work, wasting millions of clock cycles.

Q3. Explain the difference between Cycle Stealing and Block Transfer modes in DMA.

Answer: In Cycle Stealing, the DMA controller transfers data one word at a time, releasing the bus between transfers, which minimizes CPU idle time. In Block Transfer, the DMA controller keeps the bus until the entire block is moved, achieving maximum transfer rates but forcing the CPU to remain completely idle.